Gestion des événements universitaires

Équipe Warriors :
Eya Ben Saber · Eya Baccouche · Rayen Amri · Youssef Saibi · Hana Abidi · Amal Barhoumi

1. Le projet

Une plateforme web unique pour gérer tout le cycle d'un événement universitaire : inscription, réservation de salle, badge QR, contrôle d'entrée, avis et statistiques. Elle est construite en six microservices, un par membre de l'équipe.

Le problème

Les inscriptions et les présences sont suivies dans des fichiers dispersés, les salles sont réservées à la main avec un risque de double réservation, et l'entrée n'est pas contrôlée : aucun bilan de présence ni d'avis.

Hors périmètre

Paiement en ligne, diffusion vidéo et gestion des comptes, confiée à Keycloak / Better Auth (il n'y a donc pas de microservice utilisateur).

Acteurs et fonctionnalités

Participant — PARTICIPANT

- Consulter les événements publiés
- S'inscrire et recevoir un badge QR
- Donner son avis

Organisateur — ORGANISATEUR

- Créer, publier et annuler des événements
- Réserver une salle
- Affecter des intervenants
- Scanner les badges
- Consulter les statistiques

Intervenant — INTERVENANT

- Déclarer ses disponibilités
- Consulter ses affectations
- Recevoir une convocation

Admin — ADMIN

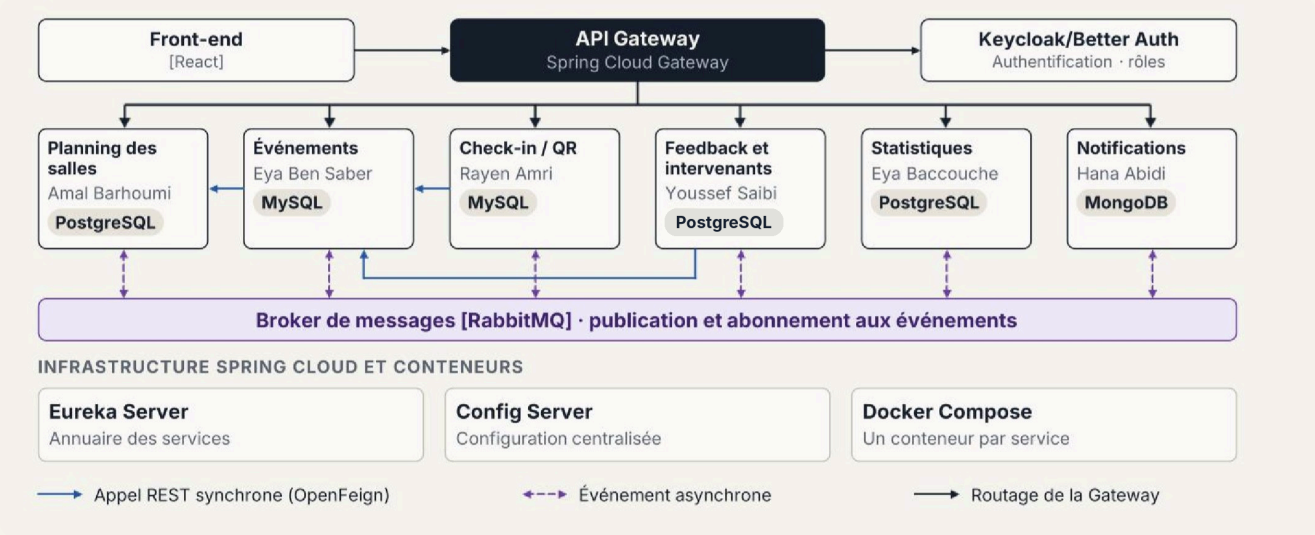
- Gérer les salles
- Superviser les événements
- Gérer les gabarits de messages

2. Architecture et technologies

Le front-end n'appelle que l'API Gateway. Elle vérifie les jetons (Keycloak / Better Auth) puis envoie chaque requête au bon microservice. Chaque service possède sa propre base de données, s'enregistre dans Eureka et récupère sa configuration auprès du Config Server.

Composant	Technologie	Rôle	Port
Microservices	Spring Boot, Java 17, Maven	Un service par étudiant, documenté avec Swagger	8081 à 8086
API Gateway	Spring Cloud Gateway	Point d'entrée unique et routage	8080
Annuaire	Eureka Server	Enregistrement et découverte des services	8761
Configuration	Config Server	Configuration centralisée	8888
Sécurité	Keycloak / Better Auth	Comptes, rôles et jetons	8180
Messages	RabbitMQ	Événements asynchrones entre services	5672
Front-end	React	Application web unique pour le groupe	5173
Conteneurs	Docker et Docker Compose	Un conteneur par service, lancement global	—

Architecture technique



Légende : flèche bleue = appel REST synchrone (OpenFeign) ; pointillés violets = événement asynchrone ; flèche noire = routage de la Gateway.

3. Les six microservices

MICROSERVICE 1 SUR 6 · EYA BEN SABER

Événements

Crée et publie les événements, gère les inscriptions et contrôle la capacité. C'est le service central.

- Suit la vie d'un événement : brouillon, publié, terminé ou annulé.
- Inscrit les participants tant qu'il reste de la place.
- Prévient les autres services à chaque étape.

Données

MySQL

Événement : id, titre, description, type, date, salleId, capacité, statut
Inscription : id, événement (@ManyToOne), participantId, statut, date
Pourquoi MySQL : données relationnelles et contrôle transactionnel de la capacité.

API REST (extrait)

Méthode	URI	Codes
POST	`/events`	201 · 400
GET	`/events?type=&statut=`	200
PUT	`/events/{id}/publish`	200 · 409
POST	`/events/{id}/registrations`	201 · 409
DELETE	`/registrations/{id}`	204 · 404

Interactions

SYNC Appelle Planning pour réserver la salle à la publication. Si Planning est arrêté, la publication est refusée et l'événement reste en brouillon.

ASYNC Publie InscriptionConfirmée, ÉvénementPublié, ÉvénementAnnulé et ÉvénementTerminé.

MICROSERVICE 2 SUR 6 · EYA BACCOUCHE

Statistiques

Agrège inscriptions, présences et avis pour produire des indicateurs. Il consomme des événements et n'appelle aucun autre service.

- Calcule l'affluence, le taux de présence et la note moyenne.
- Travaille en lecture seule, à partir des messages reçus.
- Continue de fonctionner si un autre service s'arrête.

Données

PostgreSQL

StatÉvénement : evenementId, inscrits, présents, tauxPrésence, noteMoyenne, nbAvis
Instantané : evenementId, date, inscrits, présents
Pourquoi PostgreSQL : agrégations SQL et séries temporelles.

API REST (extrait)

Méthode	URI	Codes
GET	`/stats/events/{id}`	200 · 404
GET	`/stats/events?from=&to=`	200
GET	`/stats/overview`	200
GET	`/stats/events/{id}/timeline`	200 · 404

Interactions

SYNC Aucun appel vers les autres services.

ASYNC Consomme InscriptionConfirmée, PrésenceEnregistrée, AvisEnvoyé et les événements d'état.

PANNE Si un émetteur est arrêté, le service continue avec les données déjà reçues et rattrape les messages à son retour.

MICROSERVICE 3 SUR 6 · RAYEN AMRI

Check-in / QR

Génère un badge QR pour chaque inscription et enregistre les présences à l'entrée.

- Crée un badge QR dès qu'une inscription est confirmée.
- Valide l'entrée au scan : un badge ne sert qu'une fois.
- Enregistre la présence du participant.

Données

MySQL

Badge : id, inscriptionId, événementId, participantId, code unique, statut ACTIF, UTILISÉ ou RÉVOQUÉ
Présence : id, badgeId, horodatage
Pourquoi MySQL : une contrainte d'unicité garantit un code QR unique.

API REST (extrait)

Méthode	URI	Codes
GET	`/badges/me`	200
POST	`/checkins`	201 · 404 · 409
GET	`/events/{id}/attendance`	200
PUT	`/badges/{id}/revoke`	200 · 404

Interactions

SYNC Demande à Événements si l'événement est ouvert, au moment du scan.

PANNE Si Événements est arrêté, repli sur la copie locale de l'état de l'événement.

ASYNC Consomme InscriptionConfirmée (crée le badge). Publie BadgeGénéré et PrésenceEnregistrée.

MICROSERVICE 4 SUR 6 · YOUSSEF SAIBI

Feedback et intervenants

Recueille les avis des participants, après une présence enregistrée, et gère les intervenants, jurys et leurs affectations.

- Recueille les avis des participants présents.
- Gère les profils des intervenants et des jurys.
- Les affecte à un événement comme conférencier ou jury.

Données

PostgreSQL

Avis : id, événementId, participantId, note de 1 à 5, commentaire, date
Intervenant : id, nom, fonction, disponibilités
Affectation : id, intervenantId, événementId, rôle CONFÉRENCIER ou JURY
Pourquoi PostgreSQL : base relationnelle robuste, commune avec Planning et Statistiques.

API REST (extrait)

Méthode	URI	Codes
POST	`/events/{id}/feedback`	201 · 403 · 409
GET	`/events/{id}/feedback`	200
POST	`/speakers`	201
POST	`/assignments`	201 · 409
GET	`/speakers/{id}/assignments`	200

Interactions

SYNC Appelle Événements pour valider l'événement lors d'une affectation.

PANNE Si Événements est arrêté, l'affectation est refusée et l'organisateur peut réessayer.

ASYNC Consomme PrésenceEnregistrée et ÉvénementTerminé. Publie AvisEnvoyé et IntervenantAffecté.

MICROSERVICE 5 SUR 6 · HANA ABIDI

Notifications

Envoie des messages aux utilisateurs à partir de gabarits. Il est uniquement abonné aux événements des autres services.

- Envoie confirmations, rappels, badges et convocations.
- Réagit aux messages des autres services sans jamais les appeler.
- Un envoi raté passe en échec sans bloquer personne.

Données

MongoDB

Notification : id, destinataireId, type, contenu, canal, statut, dateEnvoi
Gabarit : type, sujet, corps
Pourquoi MongoDB : contenu variable et écritures fréquentes.

API REST (extrait)

Méthode	URI	Codes
GET	`/notifications/me`	200
PUT	`/notifications/{id}/read`	200 · 404
GET	`/notifications?type=`	200
POST	`/templates`	201 · 403

Interactions

SYNC Aucun appel vers les autres services.

ASYNC Consomme InscriptionConfirmée, BadgeGénéré, ÉvénementPublié, ÉvénementAnnulé, ÉvénementTerminé et IntervenantAffecté.

PANNE Si l'envoi échoue, la notification passe au statut ÉCHEC sans bloquer les autres services.

MICROSERVICE 6 SUR 6 · AMAL BARHOUMI

Planning des salles

Gère les salles et empêche les réservations qui se chevauchent.

- Gère les salles et leurs réservations.
- Refuse tout chevauchement de créneau.
- Libère la salle quand un événement est annulé.

Données

PostgreSQL

Salle : id, nom, bâtiment, capacité
Réservation : id, salleId, événementId, début, fin, statut
Pourquoi PostgreSQL : une contrainte d'exclusion sur les plages horaires interdit les conflits.

API REST (extrait)

Méthode	URI	Codes
GET	`/rooms?minCapacity=`	200
POST	`/rooms`	201 · 403
POST	`/reservations`	201 · 409 · 422
DELETE	`/reservations/{id}`	204 · 404
GET	`/rooms/{id}/availability`	200

Interactions

SYNC Appelé uniquement par Événements, qui envoie la capacité requise dans la requête. Planning n'appelle jamais Événements.

ASYNC Consomme ÉvénementAnnulé et libère la salle.

4. Interactions entre les services

Règle : synchrone quand l'appelant a besoin de la réponse tout de suite (OpenFeign), asynchrone quand il informe les autres sans les attendre (RabbitMQ). Il y a trois appels synchrones et trois familles d'événements via RabbitMQ.

Interaction	Mode	Pourquoi, et en cas de panne
Événements → Planning : réserver la salle	SYNCHRONE	Un conflit de créneau refuse la publication. Si Planning est arrêté, l'événement reste en brouillon.
Check-in → Événements : l'événement est-il ouvert ?	SYNCHRONE	Décision prise au moment du scan. Repli sur la copie locale.
Feedback → Événements : valider l'événement	SYNCHRONE	Une affectation n'a de sens que sur un événement existant. Refus, puis nouvel essai.
InscriptionConfirmée : Événements → Check-in, Notifications, Statistiques	ASYNCHRONE	L'inscription ne dépend ni d'un envoi de message ni d'un calcul. Le message attend dans le broker.
PrésenceEnregistrée : Check-in → Statistiques, Feedback	ASYNCHRONE	Alimente l'affluence et ouvre le droit à l'avis, sans ralentir le scan.
ÉvénementPublié, Annulé, Terminé : Événements → Notifications, Planning, Feedback, Statistiques	ASYNCHRONE	Plusieurs abonnés que l'émetteur n'a pas à connaître.

Autres événements : BadgeGénéré (Check-in vers Notifications), AvisEnvoyé (Feedback vers Statistiques), IntervenantAffecté (Feedback vers Notifications).

Messages échangés (RabbitMQ, format JSON)

Événement	Émetteur	Contenu	Clé de routage
InscriptionConfirmée	Événements	inscriptionId, evenementId, participantId, date	`inscription.confirmee`
ÉvénementPublié	Événements	evenementId, titre, date, salleId	`evenement.publie`
ÉvénementAnnulé	Événements	evenementId	`evenement.annule`
ÉvénementTerminé	Événements	evenementId	`evenement.termine`
BadgeGénéré	Check-in	badgeId, inscriptionId, participantId, code	`badge.genere`
PrésenceEnregistrée	Check-in	presenceId, badgeId, evenementId, participantId, horodatage	`presence.enregistree`
AvisEnvoyé	Feedback	avisId, evenementId, participantId, note	`avis.envoye`
IntervenantAffecté	Feedback	affectationId, intervenantId, evenementId, role	`intervenant.affecte`

Un seul exchange de type **topic**, nommé **evenements**.

5. Socle commun d’équipe

En plus des six microservices métier, toute l’équipe développe le socle d’infrastructure commun basé sur Spring Cloud et Docker.

Composants communs

Composant	Rôle
API Gateway (8080)	Point d’entrée unique du front-end React (5173) et vérification des jetons (Keycloak / Better Auth).
Eureka Server (8761)	Annuaire dynamique des 6 microservices.
Config Server (8888)	Centralisation des configurations.
Docker Compose	Orchestration des conteneurs.

Routage API Gateway

Préfixe URI	Service cible	Route Eureka
`/events/**`	Événements	`lb://evenements`
`/rooms/**`, `/reservations/**`	Planning	`lb://planning`
`/badges/**`, `/checkins/**`	Check-in / QR	`lb://checkin`
`/feedback/**`, `/speakers/**`	Feedback	`lb://feedback`
`/stats/**`	Statistiques	`lb://statistiques`
`/notifications/**`	Notifications	`lb://notifications`